Project 1 Report: A Flex Lexer for Anka

BIL 463 Compiler Design · Mert Eldemir (25120205086), working alone · Path A: C + Flex · 2026-10-16

1 · Language design

Why these keywords and operators?

Anka is the language the whole semester's compiler will handle, so every token is there because a later phase (parser, type checker, code generator) needs it. Functions need `fn`, `return` and `->`. Declarations put the type after the name (`let x: int`), so the parser never has to ask C's question "is this a type or an expression?". Control flow covers both a condition loop and a counting loop (`if/else`, `while`, `for i in a..b`, `break`, `continue`). Three types and their literals (`int float bool true false`) keep type checking small. `print` is a keyword because Anka has no string type: strings may only appear as `print` arguments, which keeps P3/P4 free of string memory management. The operators are C's familiar arithmetic, comparison and logical set, plus `**` (power), compound assignment, `->` and `..`: 16 keywords and 22 operators, 13 of them multi-character.

How are multi-character operators handled (= vs == vs <=)?

Each operator is a single Flex rule (`"="`, `"=="`, `"<"`, `"<="`, …), with no hand-written lookahead. Flex combines all rules into one DFA, and its generated `yylex()` applies **maximal munch**: it follows transitions as long as it can, remembers the last accepting state it passed, and backs up to it when stuck. On `==` it passes an accepting state for `ASSIGN` after one character and one for `EQ` after two, so `EQ` wins. Tests confirm the consequences: `a-->b` → `IDENT MINUS ARROW IDENT`; `x**=2` → `POW ASSIGN` (there is no `**=`); `0..5` → `INT DOTDOT INT`, which works only because a float needs a digit after its dot.

Walk-through of the input `ifx`

Read	DFA state after it	Last accepting match remembered
`i`	could still become `if`, `in`, `int`, or any identifier	1 char → `IDENT`
`f`	both the keyword rule `"if"` and the identifier rule accept; Flex stored the rule listed **first**: `IF`	2 chars → `IF`
`x`	only the identifier rule can continue	3 chars → `IDENT`
space / end	no transition: back up to the last accepting position	result: `LINE 1: IDENT 'ifx'`

Maximal munch decides: 3 characters beat 2. Rule priority only matters on a tie: for `if` alone both rules match 2 characters, and the keyword wins because it is listed before the identifier rule. If the identifier rule were listed first, every keyword would become an `IDENT`.

2 · Testing and bugs

The suite has 7 files in `tests/` with 33 numbered cases (C1–C33; each file's header comment lists its cases), and golden outputs in `output/`. `make test` diffs the actual output against them: **7/7 pass**. The files cover: valid programs (C1–C3, 0 errors), munch and priority (C4–C12), numbers (C13–C19), strings and comments (C20–C26, C32), error recovery (C27–C30), an unterminated comment (C31), and Windows line endings (C33).

Three bugs

1. **Compiler warning inside generated code.** *Failed:* `make` printed a `-Wsign-compare` warning in `lex.yy.c`. *Why:* the scanner generated by Apple's Flex 2.6.4 compares a signed `int` with an unsigned `yy_size_t`; our `-Wextra` flags it, and generated code cannot be edited. *Fix:* compile `lex.yy.c` separately with `-Wno-sign-compare`; our own `main.c` keeps every warning. The project now builds warning-free with clang and gcc 15.
2. **Backslash at the end of an unterminated string (test C32).** *Failed:* `print("abc\` followed by a newline gave two errors, "unterminated string" and "unexpected character '\'". *Why:* the escape part of the regex, `\\.`, needs a character after the backslash, and in Flex `.` never matches a newline. So the match stopped just before the `\`, and the catch-all rule reported it separately. *Fix:* the unterminated-string rule now ends in `\\?` (an optional lone backslash), so maximal munch puts it inside the one error.
3. **Windows line endings (test C33).** *Failed:* in a `\r\n` file, an unterminated string's error message contained an invisible `\r`, which moved the terminal cursor and scrambled the output. *Why:* the string character class `[^"\\\n]` excluded `\n` but not `\r`. *Fix:* exclude both (`[^"\\\r\n]`, escapes `\\[^\r\n]`); the `\r` is then skipped as whitespace. A `.gitattributes` rule stops git from converting the test file's line endings.

For bugs 2 and 3 we rebuilt the lexer from the commit before the fix and confirmed that the new tests fail on it, so the tests really do catch these bugs.

Edge cases and surprises

- `123abc` is **not** a lexical error: munch splits it into `INT_LIT IDENT`, and rejecting it is the parser's job.
- `007` is an error but `007.5` is a valid float, as in Python 3: the integer ban exists because C reads `007` as octal, and floats never had that meaning.
- `1...5` → `INT DOTDOT`, then an error for the lone `.`, then `INT`. An unterminated string swallows the rest of its line, including `);`.
- A Turkish letter outside a string gives **two** errors, because ğ is two bytes in UTF-8 (`\xC4 \x9F`).
- The generated `lex.yy.c` declares `YY_NUM_RULES 62`. Reconciling that number with our rules showed that the spec had miscounted the keywords (15 instead of 16).

3 · AI usage and BPE comparison

AI usage

Tool: Claude Code (Anthropic, model Claude Opus 5.5), used in the terminal inside the project repository. No other AI tools were used. **How:** planning the work; designing the language through multiple-choice questions (I picked every feature); writing first versions of the spec, the lexer, the tests and the BPE script; drafting this report and the SPEC from our project logs, which I then reviewed; and explaining every file in plain language, including an interactive walkthrough page I studied before submitting. My working rules for the AI: explain every change, never run git write commands (I made every commit myself), and log every AI mistake in `docs/AI_USAGE.md`. Part A was done by hand; there the AI only acted as a tutor, with similar practice problems.

Three prompts, verbatim:

> "*let's pick it together, ask me questions and give me the interesting options*" *(designing the language)*

> "*yes, start the test suite with the 6-file plan*"

> "*before that I want from you the complete in human way understandable explanations about what you have done on each file, what is the purpose, what is the code that you've written serve for, imagine that I'm someone who is new at the compiler design so I want to truly understand what was done*"

What the AI got wrong, and how it was fixed:

- Its string rules contained bugs 2 and 3 (Section 2). They were found by its own edge-case tests, fixed, and covered by regression tests.
- Spec v0.1 claimed 15 keywords and 10 multi-character operators; the real numbers are 16 and 13. This was caught from the 62 rules in the generated `lex.yy.c` and corrected.
- Spec v0.1's float regex accepted `007.5` while `007` was an error, an inconsistency the AI didn't notice when writing it. I chose to keep the behaviour, and the spec now gives the reason.
- Its first end-of-session script ran `git commit` by itself, against my rule. It was rewritten to only print the commands, and git writes are now blocked in the tool's settings.

BPE comparison

Tokenizer: `tiktoken.encoding_for_model("gpt-4")` = `cl100k_base` (tiktoken 0.14.0). The script is `bpe/bpe_compare.py`; the full output is in `bpe/results.md` (`make bpe` regenerates it). In the table, ␣ marks a space.

Input	Our lexer	BPE (GPT-4)
All 7 test files (5,082 characters)	459 tokens (+ 27 errors)	1,681 tokens
Same content without comments or extra whitespace	459	610
`let x = 42;`	5: `LET IDENT ASSIGN INT_LIT SEMI`	6: `let · ␣x · ␣= · ␣ · 42 · ;`
`3.14159 · 1000`	`1 · 1`	4: `3 · . · 141 · 59` · 2: `100 · 0`
`ifx`	1: `IDENT(ifx)`	2: `if · x`
x@y	`IDENT`, error `@`, `IDENT`	3: `x · @ · y`
Turkish vs English sentence	(not Anka)	14 tokens for 3 words vs 4 tokens for 4 words

1. **How many tokens, and why the difference?** BPE produces 3.7× more tokens (1,681 vs 459), mostly because it also encodes comments and whitespace, which our lexer discards. On the same content the ratio drops to 1.3× (610 vs 459). The rest comes from BPE cutting numbers and rare words into frequent pieces, although it also merges some sequences we split (`-->` is a single BPE token, where we produce `MINUS ARROW`).
2. **`42`:** BPE keeps `42` as one token, but the space before it becomes a separate token. GPT-4's pre-tokenizer splits digits into groups of 1–3 and never attaches a space to a number, so `1000` becomes `100 · 0`. *Surprise:* the lecture slide shows `' 4','2'`, but none of the encodings we tested (gpt2, cl100k, o200k) does that; GPT-2 gives `' 42'`. Our lexer produces one `INT_LIT` of any length, because the parser needs the value, not its pieces.
3. **Error characters such as `@`:** our lexer reports `unexpected character '@'` with its line number, skips it and continues; the program is invalid. For BPE, `@` is just a vocabulary token: BPE has no error state at all. Anything unknown falls back to raw bytes (the hieroglyph 𓂀 becomes 4 byte tokens), so any input can be tokenized.
4. **Which is better for a parser, and for a neural network?** For a **parser**, the lexer: its tokens carry categories (`IF` vs `IDENT` vs `INT_LIT`), it guarantees keyword and identifier boundaries, and it reports errors. BPE splits `ifx` into `if · x`, so a parser would see a keyword that isn't there. For a **neural network**, BPE: a fixed vocabulary of about 100k ids, no failure on any text in any language, and frequent strings become single tokens, so sequences stay short; the network learns the meaning itself.
5. **Why does the Turkish sentence take more tokens?** `"Derleyici belirteçleri ayrıştırır"` takes 14 tokens for 3 words (4.7 per word); `"The compiler parses tokens"` takes 4 for 4 words. BPE learns its merges from training text that is mostly English, so common English words become single tokens. Turkish builds words by stacking suffixes (`belirteç+leri`, `ayrıştır+ır`), so whole words are rarely seen and get split. Letters like `ı ş ç` take 2 bytes in UTF-8 and merge less often (for example, `ç` ends up as a token by itself).